MODULE 13

ML API Design and Development

Detailed Student Study Material

Concept Explanations • Worked Examples • Exam Preparation • Expected Practicals with Answers

Module Coverage
ML engineering fundamentals, API fundamentals, REST architecture, FastAPI, Pydantic validation, Swagger UI, model serving, inference pipelines, ML API development, error handling, scalable API architecture, API security, rate limiting, API documentation, production considerations, AI-assisted API design, and hands-on model-serving tasks.

Prepared as a classroom and exam revision guide

Contents

1. Learning Outcomes
2. ML Engineering Fundamentals
3. API Fundamentals
4. FastAPI
5. Model Serving
6. ML API Development
7. API Security
8. API Documentation
9. Production Considerations
10. Thinking with AI
11. End-to-End Workflow
12. Expected Theory Questions
13. Dummy Dataset for Practicals
14. Expected Practical Questions with Complete Answers
15. Additional Practice Tasks
16. Quick Revision Sheet

1. Learning Outcomes

After completing this module, a student should be able to explain and apply the following skills:

- Distinguish machine learning from ML engineering, and model development from model serving.
- Explain what an API is, REST principles, HTTP methods, status codes, and the request-response lifecycle.
- Set up a FastAPI project, create routes, and validate data using Pydantic.
- Use FastAPI's automatic documentation (Swagger UI and ReDoc) to explore and test APIs.
- Load a trained model once, validate inputs, run inference, and format predictions as JSON.
- Build single and batch prediction endpoints with proper error handling.
- Structure an ML API so it is stateless, modular, and scalable.
- Explain and implement authentication, authorization, rate limiting, and input sanitization.
- Document an API with specifications, endpoint descriptions, and request/response examples.
- Discuss latency, scalability, reliability, monitoring, and maintainability in production.
- Use AI tools for API design and reviews while validating outputs and security.
- Build, test, document, and deploy a production-ready prediction API.

> **Important Note**
> This module moves from training models to serving them. The trained model is assumed to exist (Modules 11-12). All practicals use one running example: a heart-disease classifier served through a FastAPI application.

2. ML Engineering Fundamentals

2.1 Machine Learning vs ML Engineering

Machine Learning focuses on building a model that learns patterns from data and achieves good evaluation scores. ML Engineering focuses on making that model usable, reliable, and maintainable for real users. A model that exists only in a notebook delivers no value to anyone else.

Machine Learning	ML Engineering
Question: Can a model predict this well?	Question: Can people use this model safely and reliably?
Works in notebooks	Works in Python modules, APIs, servers, and cloud
Success = accuracy, F1, AUC	Success = uptime, latency, correct predictions in real use
Skills: statistics, algorithms, EDA	Skills: software engineering, APIs, deployment, monitoring
Output: a trained model	Output: a service other applications can call

2.2 Model Development vs Model Serving

Model development (training) creates the model. Model serving (inference) uses the saved model to answer prediction requests. They have very different requirements.

Aspect	Development (Training)	Serving (Inference)
What happens	Data, clean, train, evaluate, save	Load saved model, receive input, predict, return

Aspect	Development (Training)	Serving (Inference)
How often	Once or periodically (retraining)	Every request, possibly thousands per day
Speed needed	Minutes or hours acceptable	Milliseconds
Data	Large historical dataset	One row or a small batch at a time
Output	A model file (heart_model.joblib)	A prediction ({"prediction": 1})

2.3 Production ML Systems

Production means the system is live and used by real people. In a production ML system, the model is only a small part. Many supporting components are required around it.

- Data pipeline - brings fresh, clean data in.
- Preprocessing - applies scaling and encoding exactly as in training.
- Model - the trained estimator.
- API - lets other applications send inputs and receive predictions.
- Validation - rejects invalid inputs such as age = 500.
- Logging and monitoring - records requests and alerts when something breaks.
- Retraining - updates the model when it becomes outdated.

2.4 End-to-End ML Workflows

An end-to-end workflow covers the complete journey from business problem to a monitored live system.

1. Define the problem.
2. Collect and inspect data.
3. Perform EDA and preprocessing.
4. Train models.
5. Evaluate and select the best model.
6. Save the model or pipeline.
7. Serve it through an API.
8. Deploy the API.
9. Monitor performance.
10. Retrain when required.

Earlier modules covered steps 1 to 5. Module 13 covers steps 6 to 10.

2.5 ML Product Lifecycle

An ML product is a continuous loop rather than a straight line: build, launch, observe users, collect feedback, detect problems, improve, and release a new version.

Concept	Meaning	Example
Data drift	Live data gradually stops matching training data, so accuracy drops	A shopping model trained on 2019 data fails after buying habits change
Model versioning	Every released model has a version (v1, v2) and can be rolled back	v2 performs worse, so traffic is switched back to v1
Feedback loop	Real outcomes become new training data	Confirmed diagnoses are used to retrain

3. API Fundamentals

3.1 What Is an API?

An API (Application Programming Interface) is a contract that allows one program to request something from another program without knowing how the other program works internally.

Restaurant Analogy	API World
Customer	Client (mobile app, website, Python script)
Waiter	API
Kitchen	Server and the ML model
Menu	API documentation

Everyday examples include a weather app requesting today's temperature, "Login with Google", and UPI payments inside shopping apps.

3.2 REST Architecture

REST (Representational State Transfer) is the most common architectural style for web APIs. An API that follows REST principles is called RESTful.

Principle	Meaning	Example
Resources	Everything is identified by a URL	/patients, /patients/7
HTTP methods	Standard verbs act on resources	GET, POST, PUT, PATCH, DELETE
Stateless	Each request carries all needed information	API key sent with every request
Client-server	Frontend and backend are independent	Web and mobile apps share one API
Standard format	Data usually exchanged as JSON	{"age": 54, "chol": 239}

Method	Action	Example
GET	Read	GET /patients/7 returns patient 7
POST	Create / send data	POST /patients adds a patient; POST /predict sends features
PUT	Replace completely	PUT /patients/7 replaces all of patient 7
PATCH	Update partially	PATCH /patients/7 changes only the age
DELETE	Remove	DELETE /patients/7 removes patient 7

Why POST for Predictions?
Predictions require sending input features. POST carries data in the request body, so ML prediction endpoints normally use POST rather than GET.

3.3 Request-Response Lifecycle

Every API call is a round trip: the client sends a request, the server processes it, and a response returns.

Request Part	Example
Method	POST

Request Part	Example
URL	https://heart-api.onrender.com/v1/predict
Headers	Content-Type: application/json, X-API-Key: abc123
Body	{"age": 54, "sex": 1, "chol": 239, ...}

Response Part	Example
Status code	200
Headers	Content-Type: application/json
Body	{"prediction": 1, "probability": 0.83}

1. The client builds the request.
2. The request travels over HTTP(S).
3. The server routes it to the correct function.
4. The function validates input and runs the model.
5. The server returns a status code and JSON body.
6. The client displays the result.

Range	Meaning	Common Codes
2xx	Success	200 OK, 201 Created
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 503 Service Unavailable

3.4 API Endpoints

An endpoint is the combination of an HTTP method and a URL path. The same path with a different method is a different endpoint: GET /patients lists patients, POST /patients creates one.

URL Part	Value in https://heart-api.com/v1/patients/7?details=full
Base URL	https://heart-api.com
Version	/v1
Resource	/patients
Path parameter (which item)	7
Query parameter (options, after ?)	details=full

3.5 API Design Principles

Principle	Good	Bad
Nouns, not verbs	GET /patients	GET /getPatients
Plural, consistent names	/patients/7	/patient/7 mixed with /Users_list
Correct method	DELETE /patients/7	GET /deletePatient?id=7
Correct status codes	404 when not found	200 with {"error": "not found"}

Principle	Good	Bad
Clear error messages	{"detail": "age must be <= 120"}	{"error": "bad"}
Versioning	/v1/predict	Breaking changes without a version
Stateless	Token sent with every request	Server remembers the user

4. FastAPI

4.1 What Is FastAPI and Its Architecture?

FastAPI is a modern Python web framework for building APIs. Normal Python functions become web endpoints, and Python type hints drive validation and documentation automatically.

Component	Role
Uvicorn (ASGI server)	Listens on a port, receives HTTP requests, sends responses
Starlette (inside FastAPI)	Routing: decides which function handles which URL
Pydantic (inside FastAPI)	Validates and converts incoming and outgoing data

Request flow: Client, Uvicorn, Starlette routing, Pydantic validation, your function, JSON response.

- Automatic validation from type hints.
- Automatic interactive documentation.
- High performance with async support.
- Less boilerplate code than many alternatives.

4.2 Setting Up a Project and Running the Development Server

```
pip install fastapi "uvicorn[standard]"
# or, in a uv project:
uv add fastapi "uvicorn[standard]"
```

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def home():
    return {"message": "Hello, FastAPI is running!"}
```

```
uvicorn main:app --reload
```

Part	Meaning
main	The file main.py
app	The variable app = FastAPI() inside that file
--reload	Restart automatically when code changes (development only)

Common Error
Error loading ASGI app. Could not import module "main" means uvicorn was started from a folder that does not contain main.py. Run it from the file's folder, or use a dot path such as app.main:app.

4.3 Creating Routes with Different HTTP Methods

A route is a decorator (method and path) attached to a function.

```
@app.get("/patients/{pid}")
def get_patient(pid: int):
    if pid not in patients:
        raise HTTPException(status_code=404, detail="Patient not found")
    return patients[pid]


@app.get("/search")
def search(min_age: int = 0):
    return {pid: p for pid, p in patients.items() if p["age"] >= min_age}
```

URL	Parameter Type	How FastAPI Knows
/patients/2	Path parameter: pid = 2	{pid} appears in the route path
/search?min_age=55	Query parameter: min_age = 55	Simple argument not in the path
POST body JSON	Request body	Argument typed as a Pydantic model

Type hints also validate: with pid: int, a request to /patients/abc returns 422 automatically.

4.4 Request Handling, Response Generation, and Pydantic Validation

```
class Patient(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    age: int = Field(ge=1, le=120)      # 1 to 120 inclusive
    sex: int = Field(ge=0, le=1)        # 0 or 1
    chol: float = Field(gt=0)           # must be positive


@app.post("/patients", status_code=201)
def add_patient(patient: Patient):
    pid = max(patients, default=0) + 1
    patients[pid] = patient.model_dump()
    return {"id": pid, "patient": patients[pid]}
```

Field Rule	Meaning
ge / le	Greater / less than or equal (boundary allowed)
gt / lt	Strictly greater / less than (boundary not allowed)
min_length / max_length	String or list length limits

Body Sent	Result
{"name": "Anu", "age": 54, "sex": 0, "chol": 239}	201 Created
{"name": "Anu", "age": 200, ...}	422: age must be <= 120
{"name": "Anu", "age": "abc", ...}	422: not a valid integer

Body Sent	Result
{"name": "Anu"}	422: fields required

Returning a dict or Pydantic object produces JSON. status_code sets the success code, response_model fixes the output shape, and raising HTTPException returns an error status.

4.5 Automatic API Documentation (Swagger UI)

URL	Purpose
/docs	Swagger UI - interactive; endpoints can be tested with Try it out
/redoc	ReDoc - clean read-only documentation
/openapi.json	Raw OpenAPI specification used by both

1. Open http://127.0.0.1:8000/docs.
2. Expand an endpoint.
3. Click Try it out.
4. Edit the request body or parameters.
5. Click Execute.
6. Read the status code and response body.

5. Model Serving

5.1 Loading Trained Models

A trained model is saved to a file after training and loaded inside the API. The model must be loaded once when the server starts, never inside the prediction function.

Wrong	Right
Load inside /predict	Load once at startup
File read from disk on every request	Model stays in memory
Slow responses	Fast responses

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    if MODEL_PATH.exists():
        ml["model"] = joblib.load(MODEL_PATH)     # runs once at startup
    yield
    ml.clear()                                    # runs at shutdown

app = FastAPI(lifespan=lifespan)
```

joblib and pickle both save Python objects; joblib is preferred for scikit-learn because it stores large NumPy arrays efficiently.

5.2 Prediction Workflows

1. Receive JSON.
2. Validate it with Pydantic.
3. Convert it into the same format used in training (a one-row DataFrame with the same column names and order).
4. Call predict and predict_proba.
5. Return JSON.

Method	Returns	Example
predict(X)	Class label for each row	array([1])
predict_proba(X)	Probability of each class for each row	array([[0.17, 0.83]])

predict_proba(row)[0][1] means first row, probability of class 1 (heart disease).

5.3 Input Validation

Validation stops bad data before it reaches the model. Without it, an input like age = -5 would silently produce a meaningless prediction.

Level	Example	Handled By
Type	age must be a number	Pydantic, 422
Range	cp must be 0 to 3	Pydantic Field, 422
Required fields	All 13 features present	Pydantic, 422
Business logic	Request makes no sense	Your code, HTTPException(400)

5.4 Output Formatting

```
{"prediction": 1, "label": "heart disease", "probability": 0.83, "model_version": "1.0"}
```

Field	Why Include It
prediction	Machine-readable answer
label	Human-readable answer
probability	Confidence; clients can apply their own threshold
model_version	Identifies which model produced the result (debugging, rollback)

NumPy Types
model.predict returns numpy.int64, which cannot be converted to JSON. Convert with int() and float() before returning. A response_model also fixes the output shape in the documentation.

5.5 Inference Pipelines

Inference means making a prediction with a trained model. The inference pipeline is every step from raw input to final answer: raw input, preprocessing, model, postprocessing.

Training-serving skew occurs when preprocessing at serving time differs from training, for example when the API forgets to scale inputs. The model still runs but gives wrong predictions with no error. The fix is to save

preprocessing and the model together as one scikit-learn Pipeline.

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier(n_estimators=100, random_state=42)),
])
pipe.fit(X_train, y_train)
joblib.dump(pipe, "model/heart_model.joblib")   # pipe.predict now scales automatically
```

6. ML API Development

6.1 Building Prediction Endpoints

Endpoint	Method	Purpose
/v1/predict	POST	Prediction for one record
/v1/predict/batch	POST	Predictions for many records in one request
/v1/model-info	GET	Model version, type, and features
/health	GET	Is the API alive and the model loaded?

A batch endpoint replaces many network round trips with one request, and the model predicts all rows in a single call. The /v1 prefix allows a future /v2 without breaking existing clients.

6.2 Handling User Inputs

```
class BatchInput(BaseModel):
    records: list[HeartInput] = Field(min_length=1, max_length=100)
```

Request	Result
1-100 valid records	Processed
0 records	422 - list too short
101 records	422 - list too long (protects memory and speed)
Third record has age = -5	422 with loc ['body', 'records', 2, 'age'] (index starts at 0)

Adding model_config = ConfigDict(extra="forbid") rejects unknown fields, so a typo such as "chollesterol" produces a clear error instead of being silently ignored.

6.3 Returning Predictions

Responses must have the same structure every time. Batch results are returned in the same order as the input, together with a count.

```
df = pd.DataFrame([r.model_dump() for r in batch.records])
preds = model.predict(df)              # one call for all rows
probs = model.predict_proba(df)[:, 1]
```

6.4 Error Handling

Situation	Code	Raised By
Wrong type, out of range, unknown field, batch too big	422	Pydantic (automatic)
Valid format but logically wrong	400	Your code
Endpoint does not exist	404	FastAPI (automatic)
Model not loaded	503	Your code
Unexpected bug	500	Global exception handler

```
@app.exception_handler(Exception)
async def unhandled_error(request: Request, exc: Exception):
    logger.exception("Unhandled error on %s", request.url.path)
    return JSONResponse(status_code=500, content={"detail": "Internal server error"})
```

Why Hide Stack Traces?
Stack traces can reveal file paths, library versions, and code structure. Log full details for developers; return only a clean message to users.

6.5 Scalable API Architecture

A scalable API continues to perform well as traffic grows.

- Stateless design: no user data stored in server memory between requests, so many copies can run.
- Modular code: separate files for configuration, schemas, model loading, and routes.
- Routers: APIRouter(prefix="/v1") groups related endpoints.
- Dependency injection: Depends(get_model) shares common logic across routes.
- Configuration through environment variables instead of hard-coded values.
- Multiple worker processes in production: uvicorn main:app --workers 4 (without --reload).

File	Responsibility
config.py	Settings read from environment variables
schemas.py	Pydantic input and output models
model_loader.py	Loading and providing the model
routers/predict.py	Prediction endpoints
main.py	Creates the app and connects everything

7. API Security

Once an API is online, anyone can call it. Without protection, people can use it for free, flood it with requests, send harmful inputs, or copy the model by querying it repeatedly.

Question	Security Area	Failure Code
Who are you?	Authentication	401
What may you do?	Authorization	403
How much may you use?	Rate limiting	429

Question	Security Area	Failure Code
Is your input safe?	Input sanitization / validation	422

7.1 Authentication Awareness

Authentication verifies identity.

Method	How It Works	Best For
API key	Secret string in a header, e.g. X-API-Key	Service-to-service, simple APIs
JWT	Login once, receive signed token, send Authorization: Bearer <token>	Apps with user logins
OAuth2	A trusted provider vouches for the user	Login with Google and similar

A JWT has three dot-separated parts: header.payload.signature. The payload is only encoded, not encrypted, so it must never contain passwords. The signature proves the token has not been tampered with.

```
def verify_api_key(x_api_key: str | None = Header(default=None)) -> str:
    if x_api_key is None or x_api_key not in API_KEYS:
        raise HTTPException(status_code=401, detail="Invalid or missing API key")
    return API_KEYS[x_api_key]
```

7.2 Authorization Concepts

Authorization decides what an authenticated user is allowed to do.

	Authentication	Authorization
Question	Who are you?	What can you do?
Order	First	Second
Failure	401 Unauthorized	403 Forbidden
Example	Is this API key valid?	May this key view /v1/model-info?

Role	/v1/predict	/v1/model-info
viewer	Allowed	403 Forbidden
admin	Allowed	Allowed

Principle of least privilege: give each user only the access they actually need.

7.3 Rate Limiting Awareness

Rate limiting caps the number of requests a client may make in a time window, such as 10 per minute. Exceeding the limit returns 429 Too Many Requests.

Threat	How Rate Limiting Helps
Abuse or denial of service	Excess requests are rejected before reaching the model
Cloud cost	Usage remains predictable
Brute-force key guessing	Too few attempts allowed to succeed
Model extraction	Copying the model through many queries becomes slow

7.4 Input Sanitization

Never trust user input. Validate strictly by allowing only expected values.

- Type checks (age: int).
- Range limits (Field(ge=1, le=120)).
- String and list length limits.
- Reject unknown fields (extra="forbid").
- Limit request body size at the server or proxy.

Dangerous Pattern	Risk	Safe Alternative
User text inside an f-string SQL query	SQL injection	Parameterized queries
eval(), exec(), or shell commands on user input	Code execution	Never execute user input
Loading uploaded .pkl or .joblib files	Pickle can run code when loaded	Load only your own model files

7.5 Secure ML APIs

ML Risk	What Happens	Defence
Model extraction	Many queries used to train a copy of the model	Authentication, rate limits, rounded outputs
Out-of-range / adversarial input	Unusual inputs force wrong predictions	Strict range validation
Data leakage	Responses or errors reveal internals	Minimal responses, clean errors
Malicious model file	Tampered pickle file runs code	Load only trusted model files
Privacy	Medical data is sensitive	HTTPS, avoid logging personal details

Security Checklist
Secrets in environment variables or a .env file listed in .gitignore; HTTPS in production; clean error messages; restrictive CORS; least privilege; logging of 401, 403, and 429 events.

8. API Documentation

8.1 API Specifications

An API specification is a machine-readable description of every endpoint, parameter, request body, response, and error. FastAPI generates an OpenAPI specification automatically at /openapi.json. Swagger UI, ReDoc, client generators, and testing tools all read this specification.

8.2 Endpoint Documentation

```
@app.post("/v1/predict", response_model=HeartOutput, tags=["prediction"],
          summary="Predict heart disease for one patient", responses=ERRORS)
```

Option	Effect in /docs
title, version, description in FastAPI()	API name and introduction at the top
tags	Groups endpoints into sections
summary / docstring	Short title and longer explanation for each endpoint

Option	Effect in /docs
Field(description=...)	Explains each input field
responses	Lists error codes such as 401, 422, 429, 503
response_model	Shows the exact success response schema

8.3 Request Examples

Examples pre-fill Swagger's Try it out box so developers can test immediately.

```
model_config = ConfigDict(
    extra="forbid",
    json_schema_extra={"examples": [{
        "age": 61, "sex": 1, "cp": 0, "trestbps": 150, "chol": 300, "fbs": 0,
        "restecg": 0, "thalach": 115, "exang": 1, "oldpeak": 2.5, "slope": 1,
        "ca": 2, "thal": 3}]},
)
```

```
curl -X POST http://127.0.0.1:8000/v1/predict \
  -H "Content-Type: application/json" \
  -H "X-API-Key: viewer-key-123" \
  -d '{"age": 61, "sex": 1, "cp": 0, "trestbps": 150, "chol": 300, "fbs": 0,
       "restecg": 0, "thalach": 115, "exang": 1, "oldpeak": 2.5, "slope": 1,
       "ca": 2, "thal": 3}'
```

8.4 Response Examples

Case	Status	Example Body
Success	200	{"prediction": 1, "label": "heart disease", "probability": 1.0, "model_version": "1.0"}
Missing key	401	{"detail": "Invalid or missing API key"}
Not admin	403	{"detail": "Admin access required"}
Invalid input	422	{"detail": [{"loc": ["body", "age"], "msg": "Input should be less than or equal to 120"}]}
Too many requests	429	{"error": "Rate limit exceeded: 10 per 1 minute"}
Model missing	503	{"detail": "Model not loaded"}

8.5 Developer Usability

Good documentation lets a new developer make a successful call within a few minutes.

README Section	Contents
Overview	What the API does and who it is for
Setup	Install, train, and run commands
Authentication	Which header to send and how to obtain a key
Endpoints	Method, path, purpose
Input fields	Name, type, allowed range, meaning, units
Example request and response	A copy-paste curl or Python example
Error codes	Each code and what to do about it

9. Production Considerations

9.1 Latency Awareness

Latency is the time from sending a request to receiving the response. Report percentiles rather than only the average: p50 is the typical request, p95 and p99 show the slowest users.

Cause of Latency	Mitigation
Loading the model per request	Load once at startup
Large or complex model	Smaller model, fewer trees, model compression
Row-by-row prediction in loops	Vectorized batch prediction
Network distance	Deploy near users
Blocking work in async def	Use def for CPU-bound prediction

9.2 Scalability

Type	Meaning	Example
Vertical scaling	Bigger machine (more CPU, RAM)	Upgrade server from 2 to 8 cores
Horizontal scaling	More copies behind a load balancer	Run 4 API instances

Horizontal scaling only works when the API is stateless. Multiple worker processes use all CPU cores.

9.3 Reliability

- Health check endpoint (/health) for monitors and load balancers.
- Graceful errors with correct status codes instead of crashes.
- Timeouts on outgoing calls so one slow dependency cannot hang the API.
- Automatic restarts by the hosting platform.
- Pinned dependency versions in requirements.txt; the scikit-learn version used for serving should match the version used for training.

9.4 Monitoring

System Metrics	ML Metrics
Requests per minute	Distribution of input features over time (data drift)
Error rate (4xx, 5xx)	Distribution of predictions (e.g. share of positive predictions)
Latency p50 / p95	Accuracy once real outcomes are known
CPU and memory usage	Model version serving each prediction

9.5 Maintainability

- Clear project structure and readable code.
- Automated tests (pytest with TestClient).
- Model versioning and a documented retraining process.
- Configuration through environment variables.

- Up-to-date README and API documentation.

10. Thinking with AI

10.1 Using AI in API Work

Activity	Example Prompt
API design discussion	Suggest REST endpoints and status codes for a heart-disease prediction API.
Architecture review	Review this FastAPI project structure for scalability and maintainability.
Documentation generation	Write a README for this API including auth, examples, and error codes.
Production planning	List the monitoring metrics and reliability checks needed before launch.

10.2 Validating Generated Outputs

AI Output	What to Verify
Generated code	It runs, uses correct field names, and passes tests
Library usage	Matches current official documentation and installed versions
Status codes	Match the actual situation (401 vs 403, 422 vs 400)
Documentation	Matches real endpoints, fields, and responses
Performance claims	Measured, not assumed

10.3 Reviewing Security Concerns

- Ask explicitly: "What security problems does this code have?"
- Check for hard-coded secrets, missing authentication, and missing rate limits.
- Check that error messages do not leak internals.
- Check that user input never reaches SQL, eval, shell commands, or pickle loading.
- Treat AI output as a draft that must be reviewed, not as a final answer.

11. End-to-End Workflow

1. Train and evaluate the model.
2. Save preprocessing and model together as a Pipeline.
3. Create the FastAPI application.
4. Load the model once at startup.
5. Define Pydantic input and output schemas with validation rules.
6. Build single and batch prediction endpoints.
7. Add health and model-info endpoints.
8. Add error handling for 422, 503, and unexpected 500 errors.
9. Add authentication, authorization, and rate limiting.
10. Add documentation, examples, and a README.
11. Add request logging and automated tests.

12. Pin dependencies in requirements.txt.
13. Deploy the inference service.
14. Monitor latency, errors, and prediction distributions.
15. Retrain and release new model versions when needed.

12. Expected Theory Questions

1. What is the difference between Machine Learning and ML Engineering?

Machine learning builds models that learn from data; ML engineering makes those models usable, reliable, and maintainable in production through APIs, deployment, and monitoring.

2. Differentiate model development and model serving.

Development trains and evaluates a model offline; serving loads the saved model and returns predictions for live requests quickly and repeatedly.

3. What is an API?

An API is a contract that allows one program to request data or services from another without knowing its internal implementation.

4. What is REST? List its key principles.

REST is an architectural style for web APIs based on resources identified by URLs, standard HTTP methods, stateless requests, client-server separation, and standard formats such as JSON.

5. What does stateless mean in REST?

Each request contains all the information needed to process it; the server keeps no memory of earlier requests.

6. Explain the request-response lifecycle.

The client sends a request (method, URL, headers, body); the server routes, validates, and processes it; it returns a status code, headers, and a body.

7. Differentiate 401 and 403.

401 means the client is not authenticated; 403 means the client is authenticated but not allowed to perform the action.

8. What is the architecture of FastAPI?

Uvicorn (ASGI server) handles connections, Starlette handles routing, and Pydantic handles data validation.

9. Differentiate path and query parameters.

A path parameter is part of the URL path and identifies a resource (/patients/7); a query parameter comes after ? and provides options or filters (?min_age=40).

10. What is Pydantic used for in FastAPI?

Pydantic defines data models that validate and convert request and response data; invalid input returns 422.

11. Why should a model be loaded once at startup?

Loading reads the file from disk and is slow; loading once keeps the model in memory for fast predictions.

12. What is training-serving skew and how is it prevented?

It is a mismatch between preprocessing at training and serving time; it is prevented by saving preprocessing and model together as one Pipeline.

13. Why must NumPy values be converted before returning JSON?

NumPy types such as numpy.int64 are not JSON-serializable; int() and float() convert them to Python types.

14. Why provide a batch prediction endpoint?

It reduces network round trips and allows one vectorized model call for many records.

15. What is the role of a global exception handler?

It catches unexpected errors, logs full details for developers, and returns a clean 500 message to users.

16. What is rate limiting and why is it important for ML APIs?

It caps requests per client per time window, protecting against abuse, high cost, brute force, and model extraction; exceeding it returns 429.

17. What is model extraction?

An attack where many queries and their predictions are used to train a copy of the model.

18. What is the OpenAPI specification?

A machine-readable description of an API's endpoints, inputs, outputs, and errors; FastAPI generates it automatically and Swagger UI is built from it.

19. Differentiate vertical and horizontal scaling.

Vertical scaling uses a bigger machine; horizontal scaling runs more copies of a stateless service behind a load balancer.

20. Why are p95 and p99 latency reported?

They show the experience of the slowest users, which an average hides.

21. Why must AI-generated API code be validated?

It may use outdated libraries, wrong field names, incorrect status codes, or insecure patterns; it must be run, tested, and reviewed.

13. Dummy Dataset for Practicals

The following synthetic heart-disease dataset is used in all practicals. It has 30 patients, 13 features, and a balanced target (15 with disease, 15 without). It is small and synthetic, so it demonstrates the workflow rather than real medical performance.

Column	Meaning	Values
age	Age in years	35-71
sex	1 = male, 0 = female	0, 1
cp	Chest pain type	0-3
trestbps	Resting blood pressure (mm Hg)	110-164
chol	Serum cholesterol (mg/dl)	192-318

Column	Meaning	Values
fbs	Fasting blood sugar > 120 mg/dl	0, 1
restecg	Resting ECG result	0, 1
thalach	Maximum heart rate achieved	105-179
exang	Exercise-induced angina	0, 1
oldpeak	ST depression	0.1-3.5
slope	Slope of peak exercise ST segment	0-2
ca	Number of major vessels	0-3
thal	Thalassemia type	2, 3
target	1 = heart disease, 0 = no heart disease	0, 1

```
import pandas as pd

data = {
    "age": [44, 48, 35, 51, 57, 52, 50, 53, 61, 71, 58, 67, 48, 68, 63, 54, 52, 55, 60, 58,
37, 41, 49, 52, 54, 51, 58, 69, 54, 65],
    "sex": [0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0,
1, 0, 0],
    "cp": [1, 2, 2, 3, 1, 3, 1, 1, 0, 0, 2, 0, 2, 1, 0, 2, 3, 0, 0, 1, 2, 2, 2, 1, 0, 3, 2, 0,
1, 0],
    "trestbps": [160, 164, 118, 132, 161, 126, 110, 141, 151, 150, 154, 145, 117, 156, 133,
156, 154, 163, 110, 157, 144, 136, 153, 162, 138, 114, 149, 122, 122, 120],
    "chol": [207, 307, 230, 194, 205, 235, 228, 223, 312, 301, 260, 318, 227, 199, 218, 192,
313, 210, 242, 208, 317, 202, 252, 313, 252, 305, 305, 287, 283, 260],
    "fbs": [0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0,
1, 1, 0],
    "restecg": [0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0,
1, 1, 0, 0],
    "thalach": [146, 162, 145, 155, 155, 173, 107, 135, 123, 105, 167, 136, 146, 145, 137,
149, 151, 146, 146, 136, 145, 163, 157, 145, 128, 179, 166, 118, 107, 112],
    "exang": [1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1,
0, 0, 1],
    "oldpeak": [0.2, 0.2, 0.2, 0.1, 1.3, 0.4, 2.5, 3.5, 2.0, 1.7, 0.6, 2.7, 0.5, 2.0, 1.3,
1.0, 1.1, 2.0, 2.0, 2.4, 0.1, 0.7, 1.1, 2.2, 1.2, 0.8, 1.0, 1.8, 1.2, 3.0],
    "slope": [2, 2, 2, 2, 2, 2, 0, 1, 0, 1, 2, 0, 2, 1, 0, 1, 2, 1, 1, 1, 1, 2, 2, 1, 0, 2, 1,
1, 1, 1],
    "ca": [1, 0, 0, 0, 0, 0, 3, 1, 1, 2, 0, 3, 1, 1, 1, 1, 0, 2, 2, 3, 1, 0, 1, 3, 1, 0, 0, 3,
1, 3],
    "thal": [2, 3, 3, 2, 3, 2, 3, 3, 3, 3, 2, 3, 3, 2, 3, 2, 2, 2, 3, 3, 2, 3, 2, 2, 3, 2, 3,
3, 3, 2],
    "target": [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0,
0, 1, 1, 1],
}

df = pd.DataFrame(data)
df.to_csv("heart.csv", index=False)
print(df.head())
print("Shape:", df.shape)
print(df.isnull().sum().sum(), "missing values")
print(df["target"].value_counts())
```

14. Expected Practical Questions with Complete Answers

Setup
Create a project folder and install: pip install fastapi "uvicorn[standard]" scikit-learn pandas joblib slowapi requests pytest httpx (or uv add ... in a uv project). Run every command from the project folder.

Practical 1 - Create and Inspect the Dummy Dataset

Question: Create the heart-disease DataFrame, save it as heart.csv, and check its shape, missing values, and class balance.

Answer: run the dataset code from Section 13 as p1_dataset.py.

Expected result: Shape (30, 14), 0 missing values, and 15 rows in each target class. A balanced target means accuracy is a fair first metric.

Practical 2 - Train and Save a Model Pipeline

Question: Train a Random Forest with StandardScaler inside a Pipeline and save it with joblib.

```
from pathlib import Path

import joblib
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("heart.csv")
X = df.drop(columns=["target"])
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier(n_estimators=100, random_state=42)),
])
pipe.fit(X_train, y_train)
print("Test accuracy:", accuracy_score(y_test, pipe.predict(X_test)))

Path("model").mkdir(exist_ok=True)
joblib.dump(pipe, "model/heart_model.joblib")
print("Saved model/heart_model.joblib")
```

Explanation: the scaler and model are saved together, so the API cannot forget to scale inputs (no training-serving skew). The model/ folder is created automatically. On this tiny synthetic dataset the test accuracy is 1.0; real data will score lower.

Practical 3 - Load the Saved Model and Predict

Question: Load heart_model.joblib in a new script and predict for one patient.

```
import joblib
import pandas as pd

model = joblib.load("model/heart_model.joblib")

patient = {"age": 61, "sex": 1, "cp": 0, "trestbps": 150, "chol": 300, "fbs": 0,
           "restecg": 0, "thalach": 115, "exang": 1, "oldpeak": 2.5, "slope": 1,
           "ca": 2, "thal": 3}
row = pd.DataFrame([patient])

print("Prediction:", model.predict(row)[0])
print("Probability of disease:", round(model.predict_proba(row)[0][1], 3))
print(type(model.predict(row)[0]))
```

Expected result: Prediction 1, probability of disease close to 1.0, and type numpy.int64. The type shows why int() and float() are needed before returning JSON.

Practical 4 - Call a Public REST API with requests

Question: Send GET, POST, and a deliberately failing GET to JSONPlaceholder and print status codes.

```
import requests

r = requests.get("https://jsonplaceholder.typicode.com/posts/1")
print(r.status_code, r.headers["Content-Type"])
print(r.json())

r = requests.post("https://jsonplaceholder.typicode.com/posts",
                  json={"title": "test", "body": "hello", "userId": 1})
print(r.status_code, r.json())

r = requests.get("https://jsonplaceholder.typicode.com/posts/99999")
print(r.status_code)
```

Expected result: 200, then 201 with id 101, then 404. The title and body text is placeholder Latin (Lorem Ipsum); focus on the JSON structure and status codes, not the words.

Practical 5 - Create Your First FastAPI Application

Question: Create main.py with a root endpoint and run the development server.

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def home():
    return {"message": "Hello, FastAPI is running!"}

uvicorn main:app --reload
```

Expected result: http://127.0.0.1:8000 shows {"message": "Hello, FastAPI is running!"}.

Practical 6 - Build CRUD Routes with Path and Query Parameters

Question: Build an in-memory patients API with GET (all, one, search), POST, PUT, and DELETE.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Patients API", version="1.0")

# In-memory "database" (resets when the server restarts)
patients = {
    1: {"name": "Anu", "age": 54, "sex": 0, "chol": 239.0},
    2: {"name": "Ravi", "age": 61, "sex": 1, "chol": 210.0},
}


# ---------- Pydantic model (input rules) ----------
class Patient(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    age: int = Field(ge=1, le=120)      # 1 to 120 inclusive
    sex: int = Field(ge=0, le=1)        # 0 or 1
    chol: float = Field(gt=0)           # must be positive


# ---------- Routes ----------
@app.get("/")
def home():
    return {"message": "Patients API is running"}


@app.get("/patients")
def list_patients():
    return patients


@app.get("/patients/{pid}")
def get_patient(pid: int):
    if pid not in patients:
        raise HTTPException(status_code=404, detail="Patient not found")
    return patients[pid]


@app.get("/search")
def search(min_age: int = 0):
    return {pid: p for pid, p in patients.items() if p["age"] >= min_age}


@app.post("/patients", status_code=201)
def add_patient(patient: Patient):
    pid = max(patients, default=0) + 1
    patients[pid] = patient.model_dump()
    return {"id": pid, "patient": patients[pid]}


@app.put("/patients/{pid}")
def update_patient(pid: int, patient: Patient):
```

```
    if pid not in patients:
        raise HTTPException(status_code=404, detail="Patient not found")
    patients[pid] = patient.model_dump()
    return {"id": pid, "patient": patients[pid]}


@app.delete("/patients/{pid}")
def delete_patient(pid: int):
    if pid not in patients:
        raise HTTPException(status_code=404, detail="Patient not found")
    del patients[pid]
    return {"deleted": pid}
```

Explanation: pid is a path parameter, min_age is a query parameter, and Patient is the request body. Missing patients return 404, and successful creation returns 201.

Practical 7 - Validate Input and Read a 422 Error

Question: Send age = 200 to POST /patients and interpret the error.

```
import requests

r = requests.post("http://127.0.0.1:8000/patients",
                  json={"name": "Anu", "age": 200, "sex": 0, "chol": 239})
print(r.status_code)
print(r.json())
```

```
422
{'detail': [{'type': 'less_than_equal', 'loc': ['body', 'age'],
             'msg': 'Input should be less than or equal to 120',
             'input': 200, 'ctx': {'le': 120}}]}
```

Key	Meaning
loc	Where the error is (body, field age)
msg	What is wrong
input	The value sent
ctx	The rule that failed

Practical 8 - Test Endpoints in Swagger UI

Question: Use /docs to add, read, search, update, and delete patients.

1. Run the server and open http://127.0.0.1:8000/docs.
2. Expand POST /patients and click Try it out.
3. Enter a valid body and click Execute; confirm 201.
4. Try GET /search with min_age = 55.
5. Try PUT /patients/{pid} with pid = 999; confirm 404.
6. Open /redoc and /openapi.json and compare.

Practical 9 - Load the Model Once at Startup

Question: Create the heart API, load the model in a lifespan function, and add a public /health endpoint.

```
ml = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    if MODEL_PATH.exists():
        ml["model"] = joblib.load(MODEL_PATH)
        logger.info("Model loaded from %s", MODEL_PATH)
    else:
        logger.warning("Model file %s not found", MODEL_PATH)
    yield
    ml.clear()


@app.get("/health", tags=["system"], summary="Health check (public)")
def health():
    return {"status": "ok", "model_loaded": "model" in ml}
```

Expected result: the log shows "Model loaded" once at startup and /health returns {"status": "ok", "model_loaded": true}.

Practical 10 - Build the /v1/predict Endpoint

Question: Define HeartInput and HeartOutput schemas and return prediction, label, probability, and version.

```
def get_model():
    if "model" not in ml:
        raise HTTPException(status_code=503, detail="Model not loaded")
    return ml["model"]


@app.post("/v1/predict", response_model=HeartOutput, tags=["prediction"])
def predict(data: HeartInput, model=Depends(get_model)):
    row = pd.DataFrame([data.model_dump()])
    pred = int(model.predict(row)[0])
    prob = float(model.predict_proba(row)[0][1])
    return HeartOutput(prediction=pred, label=label_of(pred),
                       probability=round(prob, 3), model_version=MODEL_VERSION)
```

Explanation: the input becomes a one-row DataFrame with the training column names; NumPy values are converted to Python types; def is used because prediction is CPU-bound. The full HeartInput schema with all 13 fields is in Practical 20.

Practical 11 - Build a Batch Prediction Endpoint

Question: Accept up to 100 records and predict them with one model call.

```
class BatchInput(BaseModel):
    records: list[HeartInput] = Field(min_length=1, max_length=100)


@app.post("/v1/predict/batch", response_model=BatchOutput, tags=["prediction"])
def predict_batch(batch: BatchInput, model=Depends(get_model)):
    df = pd.DataFrame([r.model_dump() for r in batch.records])
    preds = model.predict(df)
    probs = model.predict_proba(df)[:, 1]
```

```
    results = [PredictionResult(prediction=int(p), label=label_of(int(p)),
                                probability=round(float(pr), 3))
               for p, pr in zip(preds, probs)]
    return BatchOutput(count=len(results), model_version=MODEL_VERSION, results=results)
```

Expected result: 2 records give count 2; 0 or 101 records give 422.

Practical 12 - Handle Errors Correctly

Question: Return 503 when the model is missing, reject unknown fields, and add a global 500 handler.

```
class HeartInput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    ...


@app.exception_handler(Exception)
async def unhandled_error(request: Request, exc: Exception):
    logger.exception("Unhandled error on %s", request.url.path)
    return JSONResponse(status_code=500, content={"detail": "Internal server error"})
```

Test: start the server with MODEL_PATH=missing.joblib and call /v1/predict; the response is 503 {"detail": "Model not loaded"}. Sending "chollesterol" returns 422 with type extra_forbidden.

Practical 13 - Add API Key Authentication

Question: Require an X-API-Key header on prediction endpoints and return 401 otherwise.

```
API_KEYS = dict(
    pair.split(":") for pair in
    os.getenv("API_KEYS", "viewer-key-123:viewer,admin-key-456:admin").split(",")
)


def verify_api_key(x_api_key: str | None = Header(default=None)) -> str:
    """Authentication: returns the caller's role, or 401."""
    if x_api_key is None or x_api_key not in API_KEYS:
        raise HTTPException(status_code=401, detail="Invalid or missing API key")
    return API_KEYS[x_api_key]
```

Add role: str = Depends(verify_api_key) to each prediction route. FastAPI maps the parameter x_api_key to the header X-API-Key. In Swagger, enter the key in the x-api-key field.

> **Secrets**
> The default keys are for local practice only. In real deployments, set API_KEYS as an environment variable or in a .env file that is listed in .gitignore and never committed.

Practical 14 - Add Role-Based Authorization

Question: Allow only admin keys to call /v1/model-info; viewer keys must receive 403.

```
def require_admin(role: str = Depends(verify_api_key)) -> str:
    """Authorization: only admins pass, others get 403."""
    if role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return role
```

```
@app.get("/v1/model-info", tags=["admin"])
def model_info(role: str = Depends(require_admin), model=Depends(get_model)):
    return {
        "model_version": MODEL_VERSION,
        "model_type": type(model[-1]).__name__,
        "features": list(HeartInput.model_fields.keys()),
    }
```

Expected result: no key gives 401, viewer key gives 403, admin key gives 200 with model_type RandomForestClassifier. model[-1] is the last step of the Pipeline.

Practical 15 - Add Rate Limiting

Question: Limit /v1/predict to 10 requests per minute per client using slowapi.

```
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)


@app.post("/v1/predict", response_model=HeartOutput, tags=["prediction"])
@limiter.limit("10/minute")
def predict(request: Request, data: HeartInput,
            role: str = Depends(verify_api_key), model=Depends(get_model)):
    ...
```

Important: the @limiter.limit decorator must be placed below the route decorator, and the function must accept request: Request. Expected result: the 11th request within a minute returns 429.

Practical 16 - Document the API

Question: Add API description, tags, summaries, field descriptions, error responses, and request examples.

```
app = FastAPI(
    title="Heart Disease Prediction API",
    version=MODEL_VERSION,
    description="Predicts heart disease risk from 13 clinical features. "
                "Send your API key in the `X-API-Key` header.",
    lifespan=lifespan,
)

ERRORS = {
    401: {"description": "Invalid or missing API key"},
    422: {"description": "Invalid input"},
    429: {"description": "Rate limit exceeded"},
    503: {"description": "Model not loaded"},
}

age: int = Field(ge=1, le=120, description="Age in years")
```

Expected result: /docs shows grouped sections (system, prediction, admin), descriptions for every field, a pre-filled example body, and the listed error codes for each endpoint.

Practical 17 - Write a README

Question: Write a README.md that lets a new developer call the API in under five minutes.

```
# Heart Disease Prediction API
Predicts heart disease risk (0/1 + probability) from 13 clinical features.

## Setup
pip install -r requirements.txt
python p1_dataset.py && python p2_train.py
uvicorn main:app --reload        # docs at http://127.0.0.1:8000/docs

## Authentication
Send header  X-API-Key: <your key>   (viewer or admin)

## Endpoints
GET  /health             public health check
POST /v1/predict         one patient          (10/minute)
POST /v1/predict/batch   up to 100 patients   (5/minute)
GET  /v1/model-info      admin only

## Errors
401 missing/invalid key | 403 not admin | 422 invalid input
429 rate limit          | 503 model not loaded
```

Practical 18 - Add Request Logging with Latency

Question: Log method, path, status code, and response time for every request.

```
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    ms = (time.perf_counter() - start) * 1000
    logger.info("%s %s -> %s (%.1f ms)", request.method, request.url.path,
                response.status_code, ms)
    return response
```

Expected log line: INFO GET /health -> 200 (1.7 ms). These logs are the starting point for latency and error-rate monitoring.

Practical 19 - Write Automated Tests

Question: Use pytest and TestClient to test success, 401, 403, 422, batch, and 429 cases.

```
import pytest
from fastapi.testclient import TestClient

from main import app, limiter

GOOD = {"age": 61, "sex": 1, "cp": 0, "trestbps": 150, "chol": 300, "fbs": 0,
        "restecg": 0, "thalach": 115, "exang": 1, "oldpeak": 2.5, "slope": 1,
        "ca": 2, "thal": 3}
```

```
VIEWER = {"X-API-Key": "viewer-key-123"}
ADMIN = {"X-API-Key": "admin-key-456"}


@pytest.fixture
def client():
    limiter.reset()
    with TestClient(app) as c:      # "with" runs startup, so the model loads
        yield c


def test_health(client):
    r = client.get("/health")
    assert r.status_code == 200 and r.json()["model_loaded"] is True


def test_predict_ok(client):
    r = client.post("/v1/predict", json=GOOD, headers=VIEWER)
    assert r.status_code == 200
    assert set(r.json()) == {"prediction", "label", "probability", "model_version"}


def test_predict_no_key(client):
    assert client.post("/v1/predict", json=GOOD).status_code == 401


def test_predict_bad_input(client):
    r = client.post("/v1/predict", json={**GOOD, "age": 200}, headers=VIEWER)
    assert r.status_code == 422


def test_unknown_field(client):
    r = client.post("/v1/predict", json={**GOOD, "chollesterol": 240}, headers=VIEWER)
    assert r.status_code == 422


def test_batch(client):
    r = client.post("/v1/predict/batch", json={"records": [GOOD, GOOD]}, headers=VIEWER)
    assert r.status_code == 200 and r.json()["count"] == 2


def test_viewer_forbidden(client):
    assert client.get("/v1/model-info", headers=VIEWER).status_code == 403


def test_admin_allowed(client):
    assert client.get("/v1/model-info", headers=ADMIN).status_code == 200


def test_rate_limit(client):
    codes = [client.post("/v1/predict", json=GOOD, headers=VIEWER).status_code
             for _ in range(11)]
    assert codes[:10] == [200] * 10 and codes[10] == 429
```

```
pytest -q
```

Expected result: 9 passed. Using TestClient inside a with block runs the startup code so the model loads. limiter.reset() clears rate-limit counters so tests do not affect one another.

Practical 20 - Complete Reference Solution - main.py

Question: Assemble all previous parts into one working production-style API.

```
"""
Heart Disease Prediction API (Module 13 final practical).
Run:  uvicorn main:app --reload        Docs: http://127.0.0.1:8000/docs
"""
import logging
import os
import time
from contextlib import asynccontextmanager
from pathlib import Path

import joblib
import pandas as pd
from fastapi import Depends, FastAPI, Header, HTTPException, Request
from fastapi.responses import JSONResponse
from pydantic import BaseModel, ConfigDict, Field
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from slowapi.util import get_remote_address

# ---------- Configuration (from environment variables) ----------
MODEL_PATH = Path(os.getenv("MODEL_PATH", "model/heart_model.joblib"))
MODEL_VERSION = os.getenv("MODEL_VERSION", "1.0")
# Format "key:role,key:role". Dev defaults only; set real keys in the environment.
API_KEYS = dict(
    pair.split(":") for pair in
    os.getenv("API_KEYS", "viewer-key-123:viewer,admin-key-456:admin").split(",")
)

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
logger = logging.getLogger("heart-api")

ml = {}


# ---------- Model loading (once at startup) ----------
@asynccontextmanager
async def lifespan(app: FastAPI):
    if MODEL_PATH.exists():
        ml["model"] = joblib.load(MODEL_PATH)
        logger.info("Model loaded from %s", MODEL_PATH)
    else:
        logger.warning("Model file %s not found", MODEL_PATH)
    yield
    ml.clear()


limiter = Limiter(key_func=get_remote_address)
```

```
app = FastAPI(
    title="Heart Disease Prediction API",
    version=MODEL_VERSION,
    description="Predicts heart disease risk from 13 clinical features. "
                "Send your API key in the `X-API-Key` header.",
    lifespan=lifespan,
)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)


# ---------- Schemas ----------
class HeartInput(BaseModel):
    model_config = ConfigDict(
        extra="forbid",
        json_schema_extra={"examples": [{
            "age": 61, "sex": 1, "cp": 0, "trestbps": 150, "chol": 300, "fbs": 0,
            "restecg": 0, "thalach": 115, "exang": 1, "oldpeak": 2.5, "slope": 1,
            "ca": 2, "thal": 3}]},
    )
    age: int = Field(ge=1, le=120, description="Age in years")
    sex: int = Field(ge=0, le=1, description="1 = male, 0 = female")
    cp: int = Field(ge=0, le=3, description="Chest pain type (0-3)")
    trestbps: float = Field(gt=0, le=300, description="Resting blood pressure (mm Hg)")
    chol: float = Field(gt=0, le=700, description="Serum cholesterol (mg/dl)")
    fbs: int = Field(ge=0, le=1, description="Fasting blood sugar > 120 mg/dl")
    restecg: int = Field(ge=0, le=2, description="Resting ECG result (0-2)")
    thalach: float = Field(gt=0, le=250, description="Maximum heart rate achieved")
    exang: int = Field(ge=0, le=1, description="Exercise-induced angina")
    oldpeak: float = Field(ge=0, le=10, description="ST depression")
    slope: int = Field(ge=0, le=2, description="Slope of peak exercise ST segment")
    ca: int = Field(ge=0, le=4, description="Number of major vessels (0-4)")
    thal: int = Field(ge=0, le=3, description="Thalassemia type (0-3)")


class BatchInput(BaseModel):
    records: list[HeartInput] = Field(min_length=1, max_length=100)


class PredictionResult(BaseModel):
    prediction: int
    label: str
    probability: float


class HeartOutput(PredictionResult):
    model_version: str


class BatchOutput(BaseModel):
    count: int
    model_version: str
    results: list[PredictionResult]
```

```
# ---------- Dependencies ----------
def get_model():
    if "model" not in ml:
        raise HTTPException(status_code=503, detail="Model not loaded")
    return ml["model"]


def verify_api_key(x_api_key: str | None = Header(default=None)) -> str:
    """Authentication: returns the caller's role, or 401."""
    if x_api_key is None or x_api_key not in API_KEYS:
        raise HTTPException(status_code=401, detail="Invalid or missing API key")
    return API_KEYS[x_api_key]


def require_admin(role: str = Depends(verify_api_key)) -> str:
    """Authorization: only admins pass, others get 403."""
    if role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return role


def label_of(pred: int) -> str:
    return "heart disease" if pred == 1 else "no heart disease"


ERRORS = {
    401: {"description": "Invalid or missing API key"},
    422: {"description": "Invalid input"},
    429: {"description": "Rate limit exceeded"},
    503: {"description": "Model not loaded"},
}


# ---------- Middleware: request logging with latency ----------
@app.middleware("http")
async def log_requests(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    ms = (time.perf_counter() - start) * 1000
    logger.info("%s %s -> %s (%.1f ms)", request.method, request.url.path,
                response.status_code, ms)
    return response


# ---------- Global error handler ----------
@app.exception_handler(Exception)
async def unhandled_error(request: Request, exc: Exception):
    logger.exception("Unhandled error on %s", request.url.path)
    return JSONResponse(status_code=500, content={"detail": "Internal server error"})


# ---------- Routes ----------
@app.get("/health", tags=["system"], summary="Health check (public)")
def health():
    return {"status": "ok", "model_loaded": "model" in ml}
```

```
@app.post("/v1/predict", response_model=HeartOutput, tags=["prediction"],
          summary="Predict heart disease for one patient", responses=ERRORS)
@limiter.limit("10/minute")
def predict(request: Request, data: HeartInput,
            role: str = Depends(verify_api_key), model=Depends(get_model)):
    row = pd.DataFrame([data.model_dump()])
    pred = int(model.predict(row)[0])
    prob = float(model.predict_proba(row)[0][1])
    return HeartOutput(prediction=pred, label=label_of(pred),
                       probability=round(prob, 3), model_version=MODEL_VERSION)


@app.post("/v1/predict/batch", response_model=BatchOutput, tags=["prediction"],
          summary="Predict for up to 100 patients", responses=ERRORS)
@limiter.limit("5/minute")
def predict_batch(request: Request, batch: BatchInput,
                  role: str = Depends(verify_api_key), model=Depends(get_model)):
    df = pd.DataFrame([r.model_dump() for r in batch.records])
    preds = model.predict(df)
    probs = model.predict_proba(df)[:, 1]
    results = [PredictionResult(prediction=int(p), label=label_of(int(p)),
                                probability=round(float(pr), 3))
               for p, pr in zip(preds, probs)]
    return BatchOutput(count=len(results), model_version=MODEL_VERSION, results=results)


@app.get("/v1/model-info", tags=["admin"], summary="Model details (admin only)",
         responses={403: {"description": "Admin access required"}, **ERRORS})
def model_info(role: str = Depends(require_admin), model=Depends(get_model)):
    return {
        "model_version": MODEL_VERSION,
        "model_type": type(model[-1]).__name__,
        "features": list(HeartInput.model_fields.keys()),
    }
```

Practical 21 - Call the Prediction API from a Client Script

Question: Write a Python client that calls /v1/predict with and without an API key.

```
import requests

BASE = "http://127.0.0.1:8000"
HEADERS = {"X-API-Key": "viewer-key-123"}

patient = {"age": 61, "sex": 1, "cp": 0, "trestbps": 150, "chol": 300, "fbs": 0,
           "restecg": 0, "thalach": 115, "exang": 1, "oldpeak": 2.5, "slope": 1,
           "ca": 2, "thal": 3}

r = requests.post(f"{BASE}/v1/predict", json=patient, headers=HEADERS, timeout=10)
print("Status:", r.status_code)
print("Body:", r.json())

r = requests.post(f"{BASE}/v1/predict", json=patient, timeout=10)   # no key
print("Without key:", r.status_code, r.json())
```

```
Status: 200
Body: {'prediction': 1, 'label': 'heart disease', 'probability': 1.0, 'model_version': '1.0'}
Without key: 401 {'detail': 'Invalid or missing API key'}
```

Practical 22 - Deploy the Inference Service

Question: Deploy the API to a cloud platform such as Render.

requirements.txt (versions pinned so training and serving use the same scikit-learn):

```
fastapi==0.162.2
uvicorn[standard]==0.54.0
scikit-learn==1.9.1
pandas==3.0.5
joblib==1.6.0
slowapi==0.1.10
```

1. Push the project (including model/heart_model.joblib, excluding .env) to GitHub.
2. Create a new Web Service on Render and connect the repository.
3. Build command: pip install -r requirements.txt
4. Start command: uvicorn main:app --host 0.0.0.0 --port $PORT
5. Add the environment variable API_KEYS with your real keys.
6. Open https://<your-service>.onrender.com/docs and test /health and /v1/predict.

Explanation: --host 0.0.0.0 accepts external traffic, $PORT is provided by the platform, and --reload is not used in production. Packaging the service as a Docker container is covered in the Containerization module.

Practical 23 - Complete Mini Project

Question: Build a production-ready heart-disease prediction service and present it.

1. Create and inspect the dataset.
2. Train and save a Pipeline.
3. Create the FastAPI app with lifespan loading.
4. Define validated input and output schemas.
5. Build single and batch prediction endpoints.
6. Add /health and admin-only /v1/model-info.
7. Handle 422, 503, and unexpected 500 errors.
8. Add API-key authentication and role-based authorization.
9. Add rate limiting.
10. Document endpoints, fields, examples, and errors.
11. Add request logging.
12. Write and pass automated tests.
13. Write the README.
14. Deploy and test the live URL.
15. State limitations: small synthetic data, in-memory API keys, no persistent monitoring.

Expected Exam Answer Pattern
For a full-mark practical, show the code, explain why each step is needed (validation, loading once, pipeline, security), show the request and the response with its status code, and interpret the result.

15. Additional Practice Tasks

1. Add a PATCH /patients/{pid} route that updates only the fields provided.
2. Use Literal[0, 1] instead of Field(ge=0, le=1) for binary fields and compare the error messages.
3. Add a configurable decision threshold as a query parameter (default 0.5) to /v1/predict.
4. Split main.py into config.py, schemas.py, model_loader.py, and routers/predict.py using APIRouter.
5. Measure average and p95 latency of 100 calls to /v1/predict using time.perf_counter.
6. Compare latency of 100 single calls versus one batch call with 100 records.
7. Replace API keys with JWT authentication using OAuth2PasswordBearer.
8. Add a /v2/predict endpoint that also returns the top three most important features.
9. Log the distribution of predictions and write a simple drift check comparing input means to training means.
10. Ask an AI tool to review main.py for security issues and verify every suggestion yourself.

16. Quick Revision Sheet

Concept	One-Line Revision
ML Engineering	Makes models usable, reliable, and maintainable in production.
Model Serving	Loads a saved model and returns predictions for live requests.
API	Contract that lets programs request services from each other.
REST	Resources, HTTP methods, stateless requests, JSON.
Endpoint	HTTP method plus URL path.
Path vs Query Parameter	Path identifies the item; query gives options after ?.
FastAPI	Starlette routing plus Pydantic validation, run by Uvicorn.
Pydantic	Validates data; invalid input returns 422.
Swagger UI	Interactive docs at /docs generated from OpenAPI.
Lifespan Loading	Load the model once at startup.
Pipeline	Saves preprocessing and model together; prevents skew.
Batch Endpoint	Many records, one request, one model call.
401	Not authenticated.
403	Authenticated but not allowed.
422	Invalid input.
429	Rate limit exceeded.
503	Service (model) unavailable.
Rate Limiting	Caps requests per client per time window.
Model Extraction	Copying a model through many queries.
OpenAPI	Machine-readable API specification.
p95 Latency	95% of requests finish within this time.

Concept	One-Line Revision
Horizontal Scaling	More stateless copies behind a load balancer.

Final Student Checklist

- I can explain ML vs ML engineering and development vs serving.
- I can explain REST, HTTP methods, and status codes.
- I can create FastAPI routes with path, query, and body parameters.
- I can validate input with Pydantic and read a 422 error.
- I can load a model once and build single and batch prediction endpoints.
- I can return clean JSON with correct status codes and handle errors.
- I can implement API keys, roles, and rate limiting.
- I can document an API with examples and write a README.
- I can test an API with pytest and TestClient.
- I can deploy an inference service and explain production considerations.
- I can use AI for API work while validating its output and security.